

TALLER 1

Análisis De Roles Y Responsabilidades En Arquitectura

Arquitectura De Software - Grupo 2

Jesús Ariel Gonzales Bonilla

Ing De Sistemas

María Sofia Aljure Herrera

Corporación Universitaria Del Huila Corhuila

Septiembre 2025

Neiva Huila

Análisis de Roles y Responsabilidades en Arquitectura

Un arquitecto de software es un profesional encargado de diseñar la estructura fundamental y las decisiones clave que guiarán el desarrollo de un sistema. Lo podríamos comparar con un arquitecto en construcción, él no pone cada ladrillo, pero es quien define cómo debe levantarse la edificación, con qué materiales, qué estilo y qué reglas debe seguir para que sea estable y funcional. Exactamente eso hace el arquitecto de software es quien define la arquitectura del sistema, selecciona tecnologías y frameworks, establece estándares de calidad, comunica la visión técnica, supervisa la implementación etc.

El arquitecto de software, así como es el encargo de diseñar la estructura fundamental de un sistema tiene algunas responsabilidades fundamentales que debe cumplir, como:

- Diseño de alto nivel
- Selección tecnológica
- Garantizar atributos de calidad
- Colaboración y liderazgo técnico
- Gestión de riesgos técnicos
- Documentación

Actividad

Escenario base

La empresa EduFlow está desarrollando una plataforma educativa en la nube, que permita gestionar:

- Cursos
- Usuarios
- Evaluaciones
- Reportes de rendimiento

El objetivo es definir una arquitectura robusta, segura y escalable. Tú has sido asignado como el Arquitecto de Software del proyecto.

1. Responsabilidades Del Arquitecto

Con base en el escenario, responde:

¿Cuáles son al menos cuatro responsabilidades clave del arquitecto de software en este proyecto?

- a. Algunas responsabilidades del arquitecto dentro de la empresa EduFlow para velar por la calidad y el rumbo en el desarrollo de la plataforma educativa son:

Definición De La Arquitectura

- Debe seleccionar el estilo arquitectónico más adecuado, considerando la escalabilidad, modularidad y mantenibilidad.
- Debe modelar la estructura de componentes y sus interacciones.
- Debe establecer la estrategia de integración entre frontend, backend y servicios externos.

Selección Tecnológica

- Debe decidir el stack tecnológico: lenguajes de programación, frameworks frontend, frameworks backend y base de datos.
- Debe elegir la infraestructura en la nube y herramientas de despliegue.
- Debe definir herramientas para CI/CD, control de versiones, gestión de dependencias y monitoreo.

Seguridad Del Sistema

- Debe diseñar mecanismos de autenticación y autorización.
- Debe definir protocolos de protección de datos sensibles.
- Debe establecer políticas contra vulnerabilidades comunes.
- Debe cumplir regulaciones de protección de datos.

Escalabilidad Y Rendimiento

- Debe asegurar que el sistema soporte cientos o miles de usuarios concurrentes en cursos y evaluaciones.
- Debe definir estrategias de balanceo de carga, caché y optimización de consultas.
- Debe diseñar un plan de elasticidad en la nube, para que la infraestructura crezca o disminuya según la demanda.

Calidad Y Mantenibilidad

- Debe establecer estándares de codificación y guías de estilo para el equipo.

- Debe promover el uso de patrones de diseño y principios SOLID para mantener el sistema flexible.
- Debe garantizar que el código sea fácilmente entendible y adaptable para futuros cambios.
- Debe definir la estructura de pruebas unitarias, de integración y de rendimiento junto con QA.

Gestión De Riesgos Técnicos

- Debe anticipar posibles problemas de escalabilidad, compatibilidad tecnológica o sobrecostos.
- Debe evaluar los riesgos de cada decisión.
- Debe proponer planes de mitigación y estrategias de contingencia.

Documentación De La Arquitectura

- Debe crear y mantener diagramas UML y documentación actualizada para que cualquier desarrollador pueda entender la arquitectura.
- Debe registrar justificación de decisiones técnicas para futuras auditorías o nuevas fases del proyecto.

¿Cómo debe interactuar con los siguientes actores?

Equipo de desarrollo (backend y frontend)

- Al equipo de desarrollo le Provee lineamientos técnicos, les entrega guías claras sobre arquitectura, patrones de diseño, estándares de codificación y herramientas que deben usarse, también Supervisa la implementación, revisa que los módulos construidos se alineen con la arquitectura definida. Si es necesario resuelve dudas técnicas, apoya a los desarrolladores cuando surgen dificultades en la integración de tecnologías, rendimiento o seguridad también ayuda a elevar el nivel técnico del equipo, promoviendo buenas prácticas y explicando decisiones arquitectónicas con mentoría y capacitación

Product Owner

- Al trabajar con el Product Owner Traduce requerimientos en soluciones técnicas el escucha las necesidades del negocio y las convierte en decisiones arquitectónicas que las hagan posibles, también Gestiona expectativas explicando cómo restricciones técnicas impactan en las funcionalidades pedidas. Debe Negociar prioridades esto ayuda al PO a decidir qué se implementa primero, considerando riesgos técnicos y dependencia entre

módulos, Comunica los riesgos advierte sobre las consecuencias de ciertas decisiones de negocio y también les garantiza que la estrategia técnica no solo resuelva lo inmediato, sino que permita la evolución futura de la plataforma educativa.

Equipo de QA

- Con el equipo de QA define criterios de calidad arquitectónica y esto establece parámetros para pruebas de rendimiento, seguridad, compatibilidad y disponibilidad, también colabora en la estrategia de pruebas lo que hace es recomendar qué escenarios deben probarse en mayor profundidad. Apoya en la automatización de pruebas, sugiere frameworks y herramientas que se integren con la arquitectura definida, monitorea métricas clave, trabaja con QA para definir indicadores de calidad como tiempo de respuesta, consumo de recursos y tasa de errores y hace retroalimentación continua ya que recibe de QA hallazgos sobre posibles cuellos de botella, vulnerabilidades o problemas de integración, y ajusta la arquitectura en consecuencia.

2. Decisiones Técnicas Críticas

Enumera cinco decisiones arquitectónicas que deben tomarse en EduFlow. Para cada una, indica:

Qué parte del sistema impacta (ejemplo: servicios, seguridad, base de datos, UI).

Qué riesgo o consecuencia podría surgir si se toma una decisión incorrecta.

Estilo arquitectónico:

La elección del estilo arquitectónico es muy importante ya que afecta la organización de los servicios y su comunicación interna; una mala decisión, como usar un monolito rígido, dificultaría la escalabilidad y mantenibilidad del sistema.

Elección de base de datos (SQL vs NoSQL):

La base de datos es quien define cómo se almacenan cursos, usuarios y evaluaciones el elegir mal podría generar bajo rendimiento en reportes y limitar la escalabilidad.

Mecanismo de autenticación y autorización (OAuth2, JWT, SSO):

los mecanismos de seguridad impactan la gestión de usuarios, si es débil, los datos personales y evaluaciones podrían quedar expuestos.

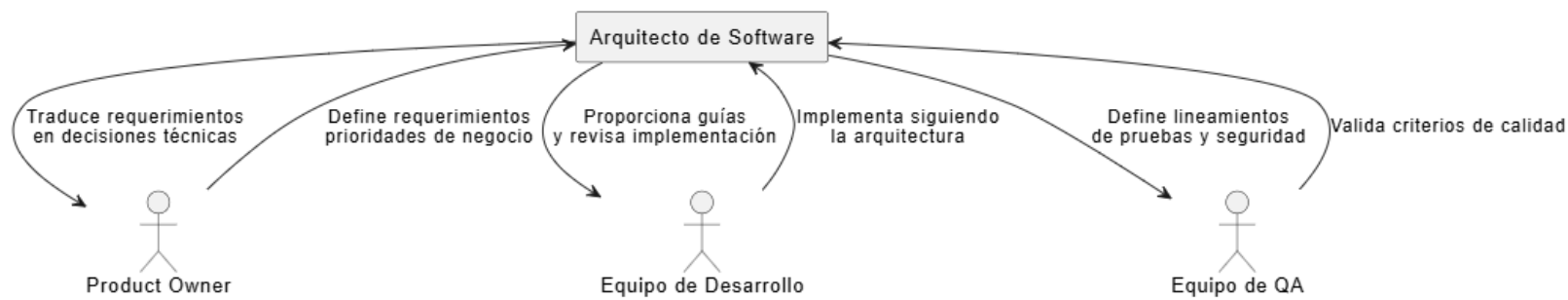
Infraestructura de despliegue (Cloud AWS/Azure/GCP, Docker, Kubernetes):

La infraestructura determina la escalabilidad y disponibilidad si un diseño llega a ser deficiente causaría caídas frecuentes y costos elevados.

Integración de analítica y reportes:

La integración de analítica afecta la generación de reportes; si se implementa mal, podrían ser lentos, imprecisos o poco útiles.

3. Diagrama PlantUML (relación del arquitecto con los otros roles técnicos en el equipo).



4. Conclusiones personales sobre la importancia del rol del arquitecto en proyectos de software a gran escala.

El arquitecto de software es clave porque da dirección y coherencia al proyecto, asegurando que las decisiones técnicas tengan sentido y que el sistema pueda crecer sin dañarse. Más que un rol técnico, es un puente entre negocio, desarrollo y calidad, ayudando a que todos trabajen alineados. En proyectos grandes, su acompañamiento marca la diferencia entre un software frágil y uno sólido y preparado para el futuro.